

Memory Architecture

Variables, Addresses & Pointer Logic in C

A Strategic 10-Page Technical Resource

1. The Concept of Memory

In C, every variable we declare is stored in the computer's RAM. To find that data later, the system assigns a **unique memory address** to that specific location.

Exam Definition: A memory address is a unique identifier for a memory location at which data is stored, represented in hexadecimal format.

Variables vs. Addresses

- **Variable:** The name we give to the data (e.g., `int age`).
- **Value:** The actual data stored (e.g., 25).
- **Address:** The location in RAM (e.g., `0x7ffee123`).

2. The Address Operator (&)

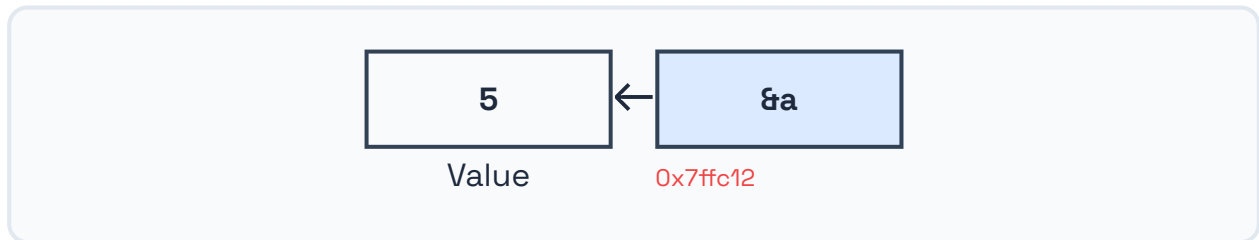
The & operator (ampersand) is known as the **Address-of Operator**. It allows us to “peek” into the system and see where our variable lives.

```
#include <stdio.h>
int main() {
    int x = 10;
    printf("Address is: %p", &x);
    return 0;
}
```

%p is the format specifier used to print memory addresses in hexadecimal.

3. Diagram Explanation

Visualizing a variable `int a = 5;`



The computer manages these hexadecimal codes to retrieve your values instantly.

4. Relation with Pointers

A **Pointer** is a special variable that doesn't store data like "10" or "true". Instead, it stores the **Memory Address** of another variable.

```
int a = 5;  
int *p = &a; // p now stores the address of a
```

The asterisk (*) in declaration tells C that p is a pointer.

5. Pointer Arithmetic

Addresses can be incremented. If an integer is 4 bytes, `ptr + 1` moves the address forward by 4 bytes.

```
int arr[3] = {10, 20, 30};  
int *p = arr;  
printf("First: %p", p);  
printf("Second: %p", p + 1);
```

6. Pointers and Arrays

In C, the name of an array acts as a constant pointer to its first element. `arr` is the same as `&arr[0]`.

Memory for arrays is always contiguous (one after another).

7. Why Addresses Matter

1. **Dynamic Memory:** Allocating RAM while the program runs.
2. **Call by Reference:** Changing a function's variable from outside.
3. **Data Structures:** Creating Linked Lists and Trees.

8. Common Mistakes

- Confusing `*p` (value) with `p` (address).
- Using an uninitialized pointer (Dangling pointer).
- Forgetting that addresses change every time you run the program.

9. Quick Summary Table

Symbol	Name	Purpose
&	Address-of	Finds the location
*	Dereference	Gets the value at address

Masterclass Concluded

Understanding memory addresses is the first step toward advanced C programming.